

Concepts

Portfolio

The Portfolio is the central hub for managing and tracking all positions across active strategies for the trading node or backtest. It consolidates position data from multiple instruments, providing a unified view of your holdings, risk exposure, and overall performance.

Currency conversion

The Portfolio supports automatic currency conversion for PnL and exposure calculations, allowing you to view results in your preferred currency. This is particularly useful when trading across multiple instruments with different settlement currencies or managing multiple accounts with different base currencies.

Supported conversions

Currency conversion is available for the following portfolio queries:

- `realized_pnl()` / `realized_pnls()` - Convert realized PnL to target currency.
- `unrealized_pnl()` / `unrealized_pnls()` - Convert unrealized PnL to target currency.
- `total_pnl()` / `total_pnls()` - Convert total PnL to target currency.
- `net_exposure()` / `net_exposures()` - Convert net exposure to target currency.

All methods accept an optional `target_currency` parameter to specify the desired output currency.

Single account behavior

When querying a single account without specifying `target_currency`, the Portfolio automatically converts values to that account's base currency:

```
# Returns exposure in the account's base currency (e.g., USD)
exposure = portfolio.net_exposures(venue=BINANCE, account_id=account_id)
```



Multi-account behavior

When querying multiple accounts simultaneously, behavior depends on whether you query all instruments (`net_exposures()`) or a single instrument (`net_exposure()`):

For `net_exposures()` (all instruments):

- **Same base currency:** Automatically converts to the common base currency.
- **Different base currencies:** Returns a dict with multiple currencies, each converted to its account's base currency. Provide `target_currency` for single-currency results.

For `net_exposure()` (single instrument across accounts):

- **Different base currencies:** Returns `None` unless you provide `target_currency`.

```
# Scenario 1: Multiple accounts, all with USD base currency
exposures = portfolio.net_exposures(venue=BINANCE)
# Returns {USD: Money(...)}
```

```
# Scenario 2: Multiple accounts with different base currencies (USD and EUR)
exposures = portfolio.net_exposures(venue=BINANCE)
# Returns {USD: Money(...), EUR: Money(...)}
```

```
# Force single currency across accounts
exposures = portfolio.net_exposures(venue=BINANCE, target_currency=USD)
# Returns {USD: Money(...)}
```



Conversion failures

When `target_currency` is provided and currency conversion fails, behavior depends on the method type:

- **Single-value methods** (`realized_pnl` , `unrealized_pnl` , `total_pnl` , `net_exposure`): Return `None` and log an error to prevent incorrect values.
- **Dict-returning methods** (`realized_pnls` , `unrealized_pnls` , `total_pnls` , `net_exposures`): Omit instruments that fail conversion but return results for successful conversions.

⚠ Exchange rate data must be available when using `target_currency` for cross-currency aggregation.

Conversion price types

When converting exposures to a target currency, the Portfolio uses different price types depending on the position composition:

- **All long positions:** Uses `BID` prices (conservative for long exposure).
- **All short positions:** Uses `ASK` prices (conservative for short exposure).
- **Mixed positions:** Uses `MID` prices (neutral when both long and short exist).

This ensures conversions reflect realistic market conditions where you would liquidate long positions at bid and cover short positions at ask. For mixed positions, mid-pricing provides a neutral valuation.

If `use_mark_xrates` is enabled in the portfolio configuration, `MARK` prices replace `MID` prices for mixed positions and general conversions.

Equity and mark-to-market

The Portfolio exposes three pull-style queries for continuous portfolio valuation. Each returns per-currency results keyed by the relevant account base currency or native settlement currency.

Method	Returns
<code>mark_values(venue, account_id)</code>	Signed MTM totals for open positions.
<code>equity(venue, account_id)</code>	Total equity combining balance and position valuation.
<code>missing_price_instruments(venue)</code>	Instruments currently flagged as unpriceable.

Longs contribute positive notional, shorts contribute negative notional. Flat positions are skipped.

Equity formula

Equity combines the account balance with open-position valuation, using a different second term depending on account type:

- **Cash and betting accounts:** `balances_total + \sum mark\_value(open positions)`.
- **Margin accounts:** `balances_total + \sum unrealized\_pnl(open positions)`.

The cash and betting path uses `mark\_values()` internally. The margin path uses the same cached unrealized PnL pipeline that powers `unrealized\_pnls()`.

Price fallback

Valuation asks `cache` for a price in this order, stopping at the first match:

1. Mark price, if `use\_mark\_prices=true` in `PortfolioConfig` and a mark price is cached.
2. Side-appropriate quote: `BID` for longs, `ASK` for shorts.
3. Last trade price.
4. Most recent cached bar close (populated when `bar\_updates=true`).

If none of the four yield a price, the position goes into the missing-price tracker and is skipped in the sum.

Base currency conversion

When `convert\_to\_account\_base\_currency=true` (the default) and the account has a `base\_currency` set, settlement-currency values are converted to the base currency using `MID` xrates from `Cache.get_xrate()`. With `use\_mark\_xrates=true`, the cached mark xrate from `Cache.get\_mark\_xrate()` is used first and falls back to `MID` if unavailable. The output dictionary then has a single key matching the base currency.

When `convert\_to\_account\_base\_currency=false`, or the account has no `base\_currency`, results are keyed by each position's native settlement currency and no xrate conversion is applied.

If xrate data is unavailable for a required conversion, that position is treated as unpriceable and flagged via the missing-price tracker rather than silently valued at a 1.0 rate.

Missing-price tracking

The tracker is a per-venue set of instrument IDs that could not be priced on the last

`mark_values()` or `equity()` call. It has two observable behaviors:

- A warning log fires once per instrument on the transition from priced to unpriced, not on every subsequent call. The next transition back to priced clears the entry so a future drop re-warns.
- When a venue goes flat (no open positions), its tracker entry is cleared so stale instruments do not remain flagged.

Call `missing_price_instruments(venue)` to inspect the current set.



If `equity()` understates what you expect, check `missing_price_instruments(venue)` before investigating the math. An empty quote, trade, and bar feed for one instrument is the most common cause of silent gaps.

Venue and account scope

`mark_values` and `equity` accept an optional `account_id` to scope the aggregation to a single account. With `account_id=None`, results aggregate across every account on the venue.

The missing-price tracker is venue-scoped. `missing_price_instruments` takes a venue only, and an account-filtered `mark_values(venue, account_id)` call does not clear the venue entry so flags raised by other accounts on the same venue survive.

Portfolio statistics

There are a variety of [built-in portfolio statistics](#) which analyse a trading portfolio's performance for both backtests and live trading.

The statistics are generally categorized as follows.

- PnLs based statistics (per currency)
- Returns based statistics
- Positions based statistics
- Orders based statistics

You can also call a trader's `PortfolioAnalyzer` and calculate statistics at any arbitrary time, including *during* a backtest, or live trading session.

Custom statistics

Custom portfolio statistics can be defined by inheriting from the `PortfolioStatistic` base class, and implementing any of the `calculate_` methods.

For example, the following is the implementation for the built-in `WinRate` statistic:

```
import pandas as pd
from typing import Any
from nautilus_trader.analysis.statistic import PortfolioStatistic

class WinRate(PortfolioStatistic):
    """
    Calculates the win rate from a realized PnLs series.
    """

    def calculate_from_realized_pnls(self, realized_pnls: pd.Series) -> Any | None:
        # Preconditions
        if realized_pnls is None or realized_pnls.empty:
            return 0.0

        # Calculate statistic
        winners = [x for x in realized_pnls if x > 0.0]
        losers = [x for x in realized_pnls if x <= 0.0]

        return len(winners) / float(max(1, (len(winners) + len(losers))))
```

These statistics can then be registered with a traders `PortfolioAnalyzer`.

```
stat = WinRate()

# Register with the portfolio analyzer
engine.portfolio.analyzer.register_statistic(stat)
```

See the `PortfolioAnalyzer` [API Reference](#) for all available methods.



Your statistic should handle degenerate inputs such as `None`, empty series, or insufficient data. Return `None` for unknown/incalculable values, or a reasonable default like `0.0` when semantically appropriate (e.g., win rate with no trades).

Returns: position vs portfolio

The analyzer tracks two distinct return series:

- **Position returns** (`analyzer.position_returns()`) measure realized return per position as a side-aware price return relative to the average open price. This reflects the instrument's price movement between entry and exit, independent of account size or leverage.
- **Portfolio returns** (`analyzer.portfolio_returns()`) measure daily percentage change in total account balance. A \$900 gain on a \$100,000 account reports roughly 0.9% for that day.

When the analyzer has account state history spanning at least two distinct calendar days, it computes portfolio returns automatically and uses them as the primary series for statistics, tearsheets, and the monthly returns heatmap. Multiple snapshots on the same day count as one day, so intra-day trading alone does not produce portfolio returns. When portfolio returns are unavailable, it falls back to position returns.

The convenience accessor `analyzer.returns()` resolves this preference: portfolio returns when present, position returns otherwise.

Multi-currency accounts

Portfolio returns require a single-currency balance history. When the account carries balances in multiple currencies, the analyzer cannot produce a single return series and falls back to position returns silently. Statistics and tearsheet charts use whichever series `returns()` resolves to.

If you need portfolio-level returns for a multi-currency account, compute them externally by converting balances to a common currency before calculating percentage changes.

Per-venue calculation

In the backtest engine, the analyzer runs per venue (`engine.pyx`). Each venue's account produces its own portfolio return series. The tearsheet aggregates across all cached accounts to produce a combined return series for multi-venue backtests.

Backtest analysis

Following a backtest run, the engine passes realized PnLs, returns, positions, and orders data to each registered statistic. Any output is then displayed in the tear sheet under the **Portfolio Performance** heading, grouped as:

- Realized PnL statistics (per currency)
- Returns statistics (for the entire portfolio)
- General statistics derived from position and order data (for the entire portfolio)

Related guides

- [Positions](#) - Position tracking within portfolios.
- [Reports](#) - Generate portfolio analysis reports.
- [Visualization](#) - Visualize portfolio performance.

[< Accounting](#)
Previous Page

[Reports >](#)
Next Page